

南 开 大 学

移 动 应 用 开 发 结 课 论 文

中文题目：课程任务智能识别与待办管理系统项目

外文题目：Course Task Intelligent Recognition and To-Do
Management System Project

学 号：2412439

姓 名：崔迪

年 级：2024 级

学 院：计算机学院

系 别：计算机科学与技术系

专 业：计算机科学与技术专业

完成日期：2026 年 6 月

指导教师：刘嘉欣 张彪 董前琨

摘 要

随着高校课程平台和教学通知渠道不断增多，学生需要同时关注雨课堂、小雅、飞书、微信群等多个入口。课程作业、实验报告、测验提醒往往分散在截图、通知文本和聊天消息中，学生需要手动识别任务内容、记录截止时间并持续跟踪完成状态，容易出现遗漏、重复记录和截止时间管理混乱等问题。传统待办应用虽然可以记录任务，但通常不能直接理解课程截图，也缺少面向学习场景的课程、平台、任务类型和优先级管理能力。

针对上述问题，本项目设计并实现了 SnapTask 课程任务智能识别与待办管理系统。系统前端采用 HarmonyOS ArkTS 与 ArkUI 声明式界面开发，使用 MVVM 模式组织页面、状态和业务逻辑；后端采用 Spring Boot 提供 REST API，使用 MySQL 存储用户、课程、OCR 记录、AI 解析记录和 Todo 任务数据；截图识别部分调用 HarmonyOS CoreVisionKit 完成本地 OCR，AI 解析部分接入 DeepSeek v4 flash 模型，将 OCR 文本抽取为结构化任务。系统支持用户登录注册、手动添加 Todo、截图识别生成多个 Todo、任务排序、任务完成勾选、已完成历史归档、横竖屏自适应界面和个人数据清理等功能。经过开发与测试，系统能够减少学生手动整理课程任务的步骤，使截图中的多条任务能够被拆分、确认、保存并持续管理，提升了移动端课程任务记录的效率和可靠性。

关键词：HarmonyOS；ArkTS；Spring Boot；MySQL；OCR 与大模型解析

Abstract

As university course platforms and teaching notification channels continue to increase, students need to simultaneously monitor multiple entry points such as Rain Classroom, Xiaoya, Feishu, and WeChat groups. Course assignments, lab reports, and quiz reminders are often scattered across screenshots, notification texts, and chat messages. Students must manually identify task content, record deadlines, and continuously track completion status, which can easily lead to issues such as missed tasks, duplicate records, and chaotic deadline management. Although traditional to-do applications can record tasks, they typically cannot directly interpret course screenshots and lack course, platform, task type, and priority management capabilities tailored to learning scenarios.

To address the above issues, this project designs and implements SnapTask, an intelligent course task recognition and to-do management system. The front end is developed using HarmonyOS ArkTS and ArkUI declarative UI, employing the MVVM pattern to organize pages, states, and business logic. The back end uses Spring Boot to provide REST APIs, with MySQL storing user, course, OCR record, AI parsing record, and to-do task data. For screenshot recognition, the system invokes HarmonyOS CoreVisionKit for local OCR, and for AI parsing, it integrates the DeepSeek v4 flash model to extract structured tasks from OCR text. The system supports user login and registration, manual addition of to-dos, generation of multiple to-dos from screenshot recognition, task sorting, task completion checkmarking, archiving of completed history, adaptive landscape and portrait UI, and personal data cleanup. Through development and testing, the system reduces the steps required for students to manually organize course tasks, enabling multiple tasks in screenshots to be split, confirmed, saved, and continuously managed, thereby improving the efficiency and reliability of mobile course task recording.

Key Words: HarmonyOS; ArkTS; Spring Boot; MySQL; OCR and Large Model Analysis

目 录

摘 要.....	I
Abstract.....	II
目 录.....	III
第一章 绪论	1
第一节 研究背景	1
第二节 研究意义	1
第三节 相关工作和现有问题	2
第四节 本文组织结构	3
第二章 工具选型	4
第一节 开发平台与语言选型	4
第二节 开发工具选型	4
第三节 前端框架选择	5
第四节 后端框架选择	5
第五节 数据库与智能能力选型	6
第三章 系统设计	7
第一节 总体设计	7
第二节 功能模块设计	8
第三节 MVVM 分层设计	9
3.3.1 视图部分详细设计	10
3.3.2 模型部分详细设计	11
3.3.3 ViewModel 控制部分详细设计	11
第四节 网络接口设计	12
第五节 数据库设计	12
第四章 系统实现	14
第一节 开发环境与运行环境	14
第二节 登录注册与首页实现	14
第三节 截图识别与 AI 解析实现	14
第四节 Todo 保存与任务详情实现	15
第五节 系统运行效果	15
第五章 总结与展望	19
第一节 全文总结	19
第二节 未来展望	19
参考文献	19

致谢	20
----------	----

第一章 绪论

第一节 研究背景

移动学习已经成为高校学生日常学习的重要组成部分。教师发布课程通知和作业要求时，常常会同时使用课程平台、即时通信工具和线下课堂说明等多种渠道。学生在完成学习任务时，不仅要关注作业内容本身，还需要记录课程名称、截止时间、提交平台、任务类型、优先等级等信息。对于课程较多、任务更新频繁的学生而言，传统手动记录方式容易造成以下问题：一是任务来源分散，需要在多个应用之间反复切换；二是截图中的文字需要人工整理，耗时较高；三是截止时间和优先级依赖个人记忆，容易遗漏；四是普通待办软件缺少课程任务语义，不能很好区分课程、平台和任务类别。

近年来，移动端 OCR 技术和大语言模型能力逐渐成熟。OCR 能够将截图中的文字转换为可处理文本，大语言模型能够进一步理解文本语义，抽取出标题、课程、截止时间、优先级等结构化信息。将 OCR 与待办管理结合，可以让学生从“手动录入任务”转向“截图识别、AI 解析、人工确认、自动归档”的流程，从而降低记录成本，提高任务管理准确性。

第二节 研究意义

本项目的意义主要体现在实际使用价值和课程开发训练两个方面。

从实际使用角度看，SnapTask 面向学生真实学习场景，尝试解决课程任务信息分散、手动录入繁琐、截止时间易遗漏的问题。系统将截图识别、AI 解析和 Todo 管理结合在同一个移动端应用中，能够把非结构化课程通知转化为可跟踪的任务列表。用户在首页查看未完成任务，在完成后勾选归档，历史页只保留已完成任务，任务生命周期更加清晰。

从课程实践角度看，本项目综合使用 HarmonyOS 前端开发、Spring Boot 后端接口、MySQL 数据库设计、网络请求、OCR 能力和大模型接口调用，覆盖了移动应用开发中常见的端到端开发流程。通过项目实现，可以加深对 MVVM 架

构、前后端分离、移动端状态刷新、数据库持久化、接口调试和横屏适配等知识的理解。

第三节 相关工作和现有问题

现有待办类应用通常具备任务新增、删除、完成标记和提醒等能力，但多数产品面向通用事务管理，对课程任务的字段设计不够细致。例如，课程名称、提交平台、任务类型和来源记录往往需要用户手动补充。另一方面，部分课程平台自带作业提醒，但提醒数据局限于单一平台，无法统一管理来自多个应用的任务。

现有已经有非常多的 Todo 管理应用，包括我校最常用的办公软件飞书也集成了待办功能。如图 1.1 所示。飞书所采用的方案正是现有大部分软件所采用的方案，非常安全，但也少了一些激进。比如缺少自动识别截图，自动添加待办的功能。



图 1.1 飞书待办示意图

OCR 工具可以识别截图文字，但识别结果通常是一整段文本，用户仍需要自行判断哪些内容是任务、哪些内容是说明。大语言模型可以进行文本理解，但如果缺少前端确认流程、数据库记录和任务状态管理，就难以形成完整可用的移动应用。因此，本项目将 OCR、大模型解析与待办系统进行整合，重点解决“识别后如何形成可管理任务”和“任务完成后如何归档”的问题。

第四节 本文组织结构

本文共分为五章。第一章介绍项目研究背景、意义和现有方案存在的问题，说明开发 SnapTask 的必要性。第二章对开发工具、开发语言、前端框架、后端框架和数据库进行选型分析，说明最终采用 HarmonyOS ArkTS、DevEco Studio、Spring Boot 和 MySQL 的原因。第三章介绍系统设计，包括总体架构、MVVM 分层、功能模块、网络接口和数据库表设计。第四章说明系统实现过程、运行效果和开发中遇到的关键问题及解决方法。第五章对项目完成情况进行总结，并对后续可改进方向进行展望。

第二章 工具选型

第一节 开发平台与语言选型

本项目目标设备为华为平板和手机，应用需要具备良好的移动端体验、系统相册访问能力、本地 OCR 能力和横竖屏适配能力。因此，前端开发平台优先考虑 HarmonyOS 原生开发。^[1]

表 2.1 部分前端开发平台对比

方案	优点	不足	适用性分析
Android 原生 Java/Kotlin	生态成熟，资料丰富， 适合 Android 设备	与 HarmonyOS 设备原生 能力结合不如 ArkTS 直 接	不作为本项目首 选
Web/H5	开发成本低、跨平台部 署方便	相册、OCR、系统级体 验和离线能力受限制	适合轻量展示、 不适合本项目核 心需求
HarmonyOS ArkTS	与鸿蒙设备适配好， ArkUI 声明式开发效率 高，可调用系统能力	对语法规则和状态管理 要求较高	符合课程要求和 设备环境，作为 最终方案

最终选择 ArkTS 作为前端主要开发语言。ArkTS 在 TypeScript 基础上加强了静态类型约束，适合构建结构清晰、状态明确的移动端应用。项目中使用 ArkUI 组件完成页面布局，并结合 HarmonyOS 系统 Kit 实现网络访问、相册选择和本地文字识别。^[2]

第二节 开发工具选型

表 2.2 部分开发工具对比

工具	优点	不足	选择结果
DevEco Studio	官方 HarmonyOS 开发工 具，支持 ArkTS、预览、 签名、构建和真机调试	对环境配置要求较高	前端主开发 工具
Eclipse	Java 课程环境常见，基础 功能完整，能够满足 Spring Boot 后端开发与运 行调试需要	开箱体验偏弱，很多功能要 靠插件和配置堆出来；界 面、智能提示、重构体验通 常不如 IDEA 顺滑。	后端主开发 工具
IntelliJ	Java 与 Spring Boot 开发	更吃内存和性能，Ultimate	作为对比方

工具	优点	不足	选择结果
IDEA	体验好，插件生态完善	版很多高级功能是付费订 阅；免费 Community 版功 能比 Ultimate 少。	案，不作为 本项目主要 工具

前端采用 DevEco Studio 进行开发、构建和真机安装。后端 Spring Boot 项目采用 Eclipse 进行开发与运行调试，并可配合命令行 Maven 完成依赖管理和打包。考虑到课程环境与项目规模，Eclipse 已能够满足后端接口开发、数据库连接配置和服务启动测试需求；前端则主要关注 ArkTS 页面开发与移动端体验。

第三节 前端框架选择

前端采用 HarmonyOS ArkUI 声明式 UI 框架。ArkUI 通过组件嵌套描述页面结构，适合快速搭建移动端页面。与传统命令式 UI 相比，声明式 UI 更适合与状态绑定结合：当任务列表、登录状态、当前页面或历史选中项发生变化时，界面能够根据状态重新渲染。^[3]

项目采用 MVVM 思想组织前端结构：View 负责页面展示，ViewModel 负责状态变化和业务流程，Model 负责数据模型、网络访问和系统能力封装。这种组织方式可以避免所有逻辑堆叠在页面中，使识别、解析、保存、删除、完成归档等流程更加清晰。

第四节 后端框架选择

后端需要提供用户登录、任务增删改查、OCR 记录保存、AI 解析结果保存和数据清理等接口。可选方案包括原生 Servlet、Spring MVC、Spring Boot 等。

表 2.3 部分后端框架对比

方案	优点	不足	选择结果
原生 Servlet	原理清晰，依赖少	配置繁琐，开发效率低	不适合快速实现完整业务
Spring MVC	适合构建 Web 接口，结构成熟	需要较多工程配置	可行但不够便捷
Spring Boot	自动配置完善，REST API 开发效率高，便于打包运行	需要理解框架约定	最终采用

本项目后端采用 Spring Boot 3.1.9，配合 MySQL Connector、JdbcTemplate 和

项目中的 MyBatis-Plus 依赖完成数据库访问与扩展配置。Spring Boot 能够快速暴露 REST 接口，适合前后端分离的移动应用开发模式。

第五节 数据库与智能能力选型

数据库选择 MySQL，原因是 MySQL 安装方便、SQL 语法成熟、适合课程项目中存储关系型数据。用户、课程、任务、OCR 记录和 AI 解析记录之间存在明确关联，使用关系型数据库便于维护数据一致性。

截图识别能力选择 HarmonyOS CoreVisionKit 的文字识别能力，能够直接在平板本地对图片进行 OCR。AI 解析能力选择 DeepSeek deepseek-v4 flash 模型，用于把 OCR 文本解析为多个结构化任务。

第三章 系统设计

第一节 总体设计

SnapTask 采用前后端分离架构。前端运行在 HarmonyOS 平板或手机上，负责用户交互、截图选择、本地 OCR、页面状态展示和网络请求；后端运行在电脑 Spring Boot 服务中，负责业务接口、数据库访问和 DeepSeek 调用；MySQL 负责持久化存储用户、课程、任务和识别记录。

系统整体流程可以概括为：用户登录后进入首页查看未完成任务；用户可以手动新增任务，也可以选择课程截图进行 OCR 识别；OCR 文本提交到后端后，由 DeepSeek 或本地规则解析为结构化任务；用户在确认页逐条检查并保存；任务保存后进入首页待办列表；用户完成任务后点击勾选框，任务状态变为已完成并从首页移入历史页。

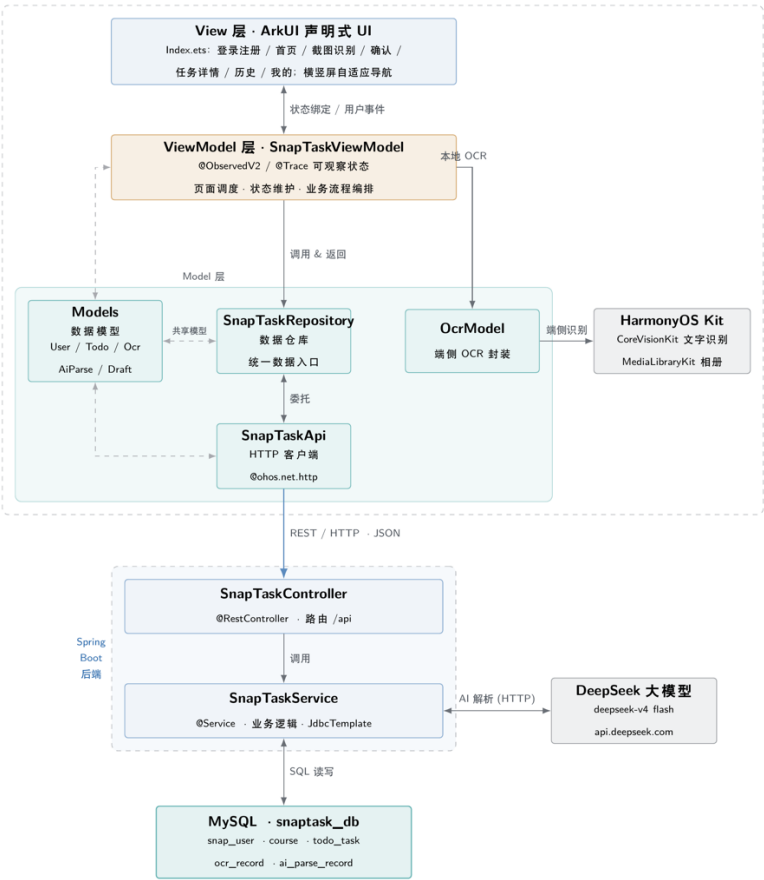


图 3.1 总体架构示意图

第二节 功能模块设计

系统围绕课程任务从创建、管理到完成归档的完整流程进行功能划分。首先是用户模块，该模块负责注册、登录、退出登录以及当前用户信息展示。由于系统需要保存用户的课程、任务和识别记录，因此用户模块是其他业务模块的基础。通过用户编号区分不同账号的数据，可以保证每个用户只查看和操作自己的任务内容。

首页任务模块是用户进入系统后的主要工作区。该模块负责展示当前未完成任务，并统计今日待办、即将截止、已逾期和已完成任务数量。首页只保留仍需处理的任务，使用户能够快速了解当前学习任务压力。任务卡片中展示标题、课程、任务类型、提交平台和截止时间等关键信息，便于用户在较短时间内判断任务优先级。

手动添加模块用于处理非截图来源的任务。用户可以直接填写任务标题、课程名称、截止时间、任务类型、提交平台、优先级和任务说明。该模块保证系统不仅依赖 OCR 和 AI，也能够覆盖课堂口头通知、线下作业要求或用户临时想到的学习事项。

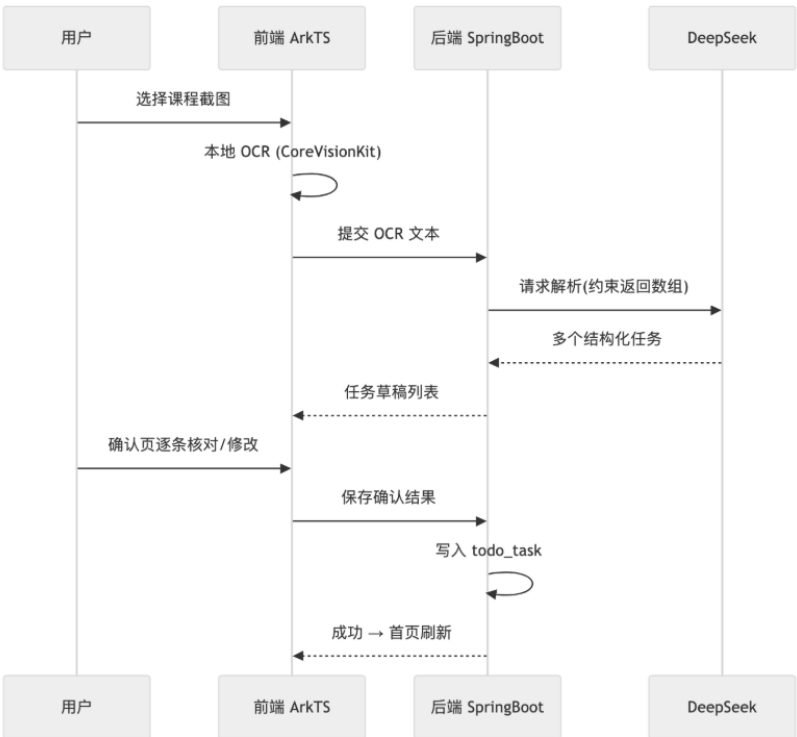


图 3.2 工作流程示意图

截图识别模块是本项目的核心特色功能。用户选择本机课程截图后，前端先调用本地 OCR 能力识别图片文字，再将 OCR 文本提交给后端进行 AI 解析。系统会把截图中的课程任务拆分为一个或多个结构化 Todo 草稿，用户可以逐条确认和修改。该模块的目标是减少手动录入成本，提高从课程截图到任务列表的转换效率。

任务详情模块用于展示单个任务的完整信息，包括标题、课程、截止时间、任务类型、提交平台、优先级、来源和任务说明。用户可以通过详情页检查任务内容是否准确，并对无效任务执行删除操作。这样既保留了首页的简洁性，又提供了进一步查看任务细节的入口。

完成归档模块负责管理任务生命周期的结束阶段。用户在首页点击任务左侧的勾选框后，系统将任务状态更新为已完成，并将其从首页移入历史页。历史页只保存已完成任务，点击不同历史任务时会显示对应的完成详情。该设计使“未完成任务”和“已完成任务”边界清晰，便于用户回顾已经处理过的课程事项。

我的模块主要负责个人信息、系统设置和数据管理。该模块展示当前账号、任务总数、已完成数量、未完成数量等信息，并提供 DeepSeek API Key 设置入口和清理本人业务数据功能。通过该模块，用户可以完成与账号和数据相关的管理操作。

横屏适配模块用于提升平板横屏场景下的使用体验。系统在横屏时将竖屏底部导航调整为左侧导航，并根据页面内容重新安排列表、表单和详情区域的位置。各页面内容区域保持顶部对齐，避免简单拉伸造成空间浪费或阅读不自然的问题。

第三节 MVVM 分层设计

系统前端采用 MVVM 开发模式，将应用划分为模型、视图和控制状态的 ViewModel 三个部分。虽然 MVVM 中严格意义上的控制层称为 ViewModel，但在本项目中它承担了页面调度、状态维护和业务流程控制的职责。

模型层负责定义数据结构和封装数据来源，包括用户会话、课程、任务草稿、Todo 任务、OCR 记录、AI 解析记录和接口响应格式等。模型层还封装了网络请求和 OCR 能力，使页面不需要直接处理 HTTP 细节或系统文件读取细节。

视图层负责展示页面和接收用户操作，包括登录注册页、首页、截图识别页、确认页、手动添加页、任务详情页、历史页和我的页面。视图层尽量只描述界面结构和交互入口，不直接处理复杂业务。

ViewModel 层负责连接模型和视图，维护当前页面、登录状态、任务列表、已完成任务列表、当前任务、当前解析结果、加载状态和提示信息。用户点击按钮后，ViewModel 调用模型层完成网络请求或 OCR 操作，再把结果转换为页面可展示的状态。



图 3.3 MVVM 分层示意图

3.3.1 视图部分详细设计

登录注册页采用居中的表单布局，用户可以在登录和注册模式之间切换。登录成功后进入首页，首页展示统计卡片、排序设置和未完成任务列表。任务卡片左侧设计待勾选框，点击后直接完成任务并移入历史。

截图识别页包含图片选择区域、OCR 原文区域和 AI 解析入口。用户选择截图后，系统显示图片状态并允许用户启动本地 OCR。OCR 得到文本后，用户可以直接修改文本，再提交给 AI 解析。

确认页用于承接 AI 解析结果。如果截图中识别出多个任务，确认页提供上一条、下一条和一键保存全部的交互。每个任务都可以修改标题、课程、截止时间、任务类型、提交平台、优先级和说明，避免 AI 解析结果直接入库造成错误。

历史页不再保存“识别历史”，而是专门保存已完成任务。左侧或上方展示已

完成任务列表，点击不同任务后，完成详情区域展示该任务的标题、课程、类型、平台、截止时间、完成时间、优先级和说明。横屏时历史页采用左右分栏，左侧为完成列表，右侧为详情区域。

界面视觉上采用莫兰迪色系，主要颜色包括浅米绿、浅棕、柔和粉红、灰蓝和浅黄色。整体风格保持简约、柔和和可阅读，避免过强对比造成视觉疲劳。

3.3.2 模型部分详细设计

模型层围绕业务对象进行设计。用户对象保存用户编号、用户名和时间信息；课程对象保存课程名称、教师和备注；Todo 任务对象保存标题、课程、描述、任务类型、平台、截止时间、优先级、状态、来源和时间信息；OCR 记录对象保存图片路径、OCR 文本、识别状态和 AI 状态；AI 解析记录对象保存大模型原始响应、解析后的任务字段、解析状态和是否已生成 Todo。

模型层还负责屏蔽数据来源差异。对于网络数据，前端通过统一 API 类访问后端接口，接口返回后再转换为前端模型对象。对于 OCR 数据，前端通过 OCR 模型完成图片选择、图片读取、PixelMap 创建、文字识别和资源释放，使页面只关心最终识别出的文本。

3.3.3 ViewModel 控制部分详细设计

ViewModel 是前端业务流程的核心。它保存当前屏幕状态，用于在登录页、首页、识别页、确认页、手动页、详情页、历史页和我的页面之间切换。它还保存加载状态和提示信息，用于反馈登录失败、后端连接失败、DeepSeek 调用失败、保存成功等结果。

在任务管理方面，ViewModel 从后端加载全部任务后，会按状态拆分为未完成任务和已完成任务。未完成任务展示在首页，已完成任务展示在历史页。用户点击完成勾选框后，ViewModel 调用后端更新任务状态，然后重新加载任务列表，使首页和历史页保持一致。

在 AI 解析方面，ViewModel 负责创建 OCR 记录、请求后端解析、保存解析结果列表、维护当前确认的任务下标，并在用户修改字段时同步更新任务草稿。多任务截图解析时，ViewModel 需要保证不同任务之间的草稿互不覆盖。

第四节 网络接口设计

前后端通过 HTTPJSON 进行通信。前端统一访问后端 /api 路径下的接口。接口返回格式包含状态码、提示信息和数据对象，便于前端统一处理成功、失败和异常情况。

主要接口设计如下。

表 3.1 网络接口设计

接口类别	功能说明
用户接口	注册、登录、获取个人信息
Dashboard 接口	统计今日待办、即将截止、逾期、已完成和任务总数
课程接口	查询课程、保存课程
Todo 接口	查询任务、查看详情、新增任务、更新任务状态、删除任务
OCR 接口	保存 OCR 原文、查询 OCR 记录、查询识别详情
AI 接口	解析 OCR 文本、确认 AI 解析结果并生成 Todo
数据接口	清理当前用户的业务数据

第五节 数据库设计

数据库采用 MySQL，库名为 snaptask_db。主要表结构设计如下。

表 3.2 数据库表结构设计

表名	存储内容	主要字段含义
snap_user	用户账号信息	用户编号、用户名、密码、创建时间、更新时间
course	课程信息	用户编号、课程名称、教师名称、颜色、备注
todo_task	Todo 任务信息	用户编号、课程编号、标题、描述、类型、平台、截止时间、优先级、状态、来源
ocr_record	截图 OCR 识别记录	用户编号、图片路径、OCR 文本、识别状态、AI 状态、生成任务信息
ai_parse_record	AI 解析记录	OCR 记录编号、原始 AI 响应、解析后的任务字段、是否生成 Todo

用户和任务通过用户编号关联，课程和任务通过课程编号关联，OCR 记录和 AI 解析记录通过 OCR 记录编号关联，AI 解析记录和最终 Todo 之间通过生成标记和来源编号建立联系。这样的设计既能支持用户独立数据，又能保留从截图到任务生成的来源追踪。

第六节 任务状态与历史归档设计

任务状态主要包括待完成、已逾期和已完成。系统在加载任务时会检查截止时间，将超过截止时间且未完成任务更新为已逾期。首页只展示未完成任务，使用户聚焦当前需要处理的内容。任务完成后，用户通过勾选框将状态改为已完成，任务随即从首页消失并进入历史页。

历史页只展示已完成任务，使用户更容易理解：首页代表当前待办，历史页代表已完成归档。点击历史页中不同任务时，系统会刷新完整完成详情，保证列表选中项和详情内容一致。

第四章 系统实现

第一节 开发环境与运行环境

前端开发环境为 DevEco Studio，使用 ArkTS 和 ArkUI 开发 HarmonyOS 应用。应用支持 phone 和 tablet 设备类型，并在华为平板上进行真机测试。前端需要申请网络权限，用于访问本地局域网中的 Spring Boot 服务。

后端开发环境为 Java 17 和 Spring Boot 3.1.9，使用 Maven 管理依赖。数据库为 MySQL，连接地址为本地 snaptask_db 数据库。后端服务运行后，平板和电脑需要处于同一 Wi-Fi 环境，前端通过电脑局域网 IP 访问后端接口。

第二节 登录注册与首页实现

系统启动后进入登录页，用户输入账号和密码后请求后端登录接口。登录成功后，前端保存用户会话并加载工作区数据，包括 Dashboard 统计和任务列表。首页通过统计卡片展示今日待办、即将截止、已逾期和已完成数量，并在下方展示当前未完成任务。

首页任务卡片包含标题、课程、任务类型、平台、截止时间和状态。左侧勾选框用于快速完成任务，点击后系统调用后端状态更新接口，将任务标记为已完成。状态更新成功后，首页重新加载未完成任务列表，该任务从首页移除。

第三节 截图识别与 AI 解析实现

截图识别流程分为四步。第一步，用户从本机相册选择课程任务截图。第二步，前端调用 HarmonyOS 本地 OCR 能力，将图片转换为文本。第三步，用户检查或修改 OCR 原文后提交给后端。第四步，后端根据提供的 DeepSeek API Key 调用 deepseek-v4 flash。

当截图中包含多个任务时，后端要求 AI 返回任务数组，并将每个任务保存为独立的 AI 解析记录。前端确认页按照任务下标逐条展示，用户可以分别修改字段。这样可以避免多个任务被错误合并成一个 Todo，也可以避免不同任务共

用同一个标题或截止时间。

第四节 Todo 保存与任务详情实现

手动添加和 AI 确认生成最终都会进入 Todo 保存流程。用户保存前至少需要填写任务标题。保存成功后，后端会根据课程名称自动维护课程记录，并将任务写入 Todo 表。前端随后刷新任务列表和统计信息，使首页数据保持最新。

任务详情页用于查看任务完整信息，包括标题、课程、任务类型、平台、截止时间、优先级、来源和创建时间。根据当前需求，详情页中有删除功能。

第五节 系统运行效果

如下图 4.1 所示，在登陆页面可以输入账号密码进行登录，也可以点击注册进行注册，注册之后的个人信息将被存入数据库。

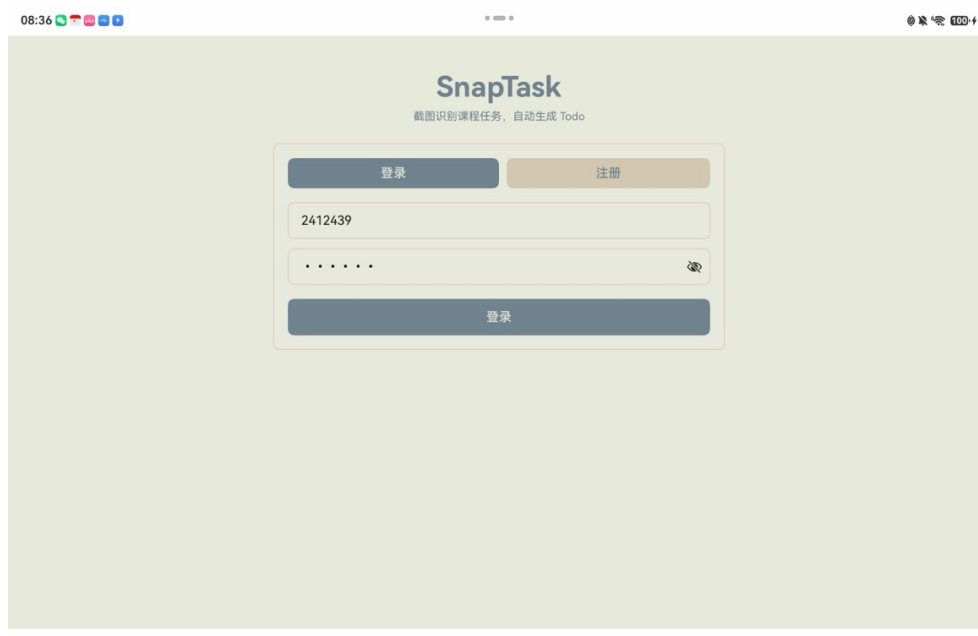


图 4.1 登陆页面运行效果

如下图 4.2 所示，在首页可以查看今日待办，即将截止，已逾期，已完成等的数量，还可以快捷的勾选任务，存入历史。也可以点击对应的任务查看详细信息。



图 4.2 首页实现效果

如下图 4.3，在识别页可以上传截图，先使用本地 OCR，再调用 deepseek-v4 flash 模型进行处理，快速得到任务待办。

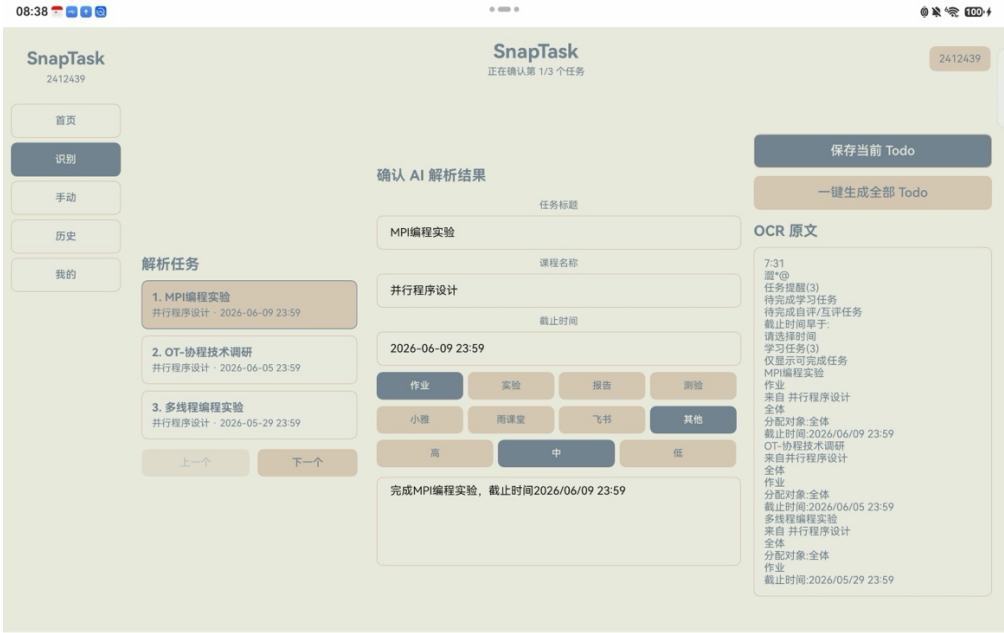


图 4.3 识别页效果

如下图 4.4 所示，可以在手动添加页面，手动写待办的具体事项，算是保留的基础功能。



图 4.4 手动添加页效果图

如下图 4.5 所示，可以在历史页面查看已完成的任务列表并查看对应任务的具体信息。

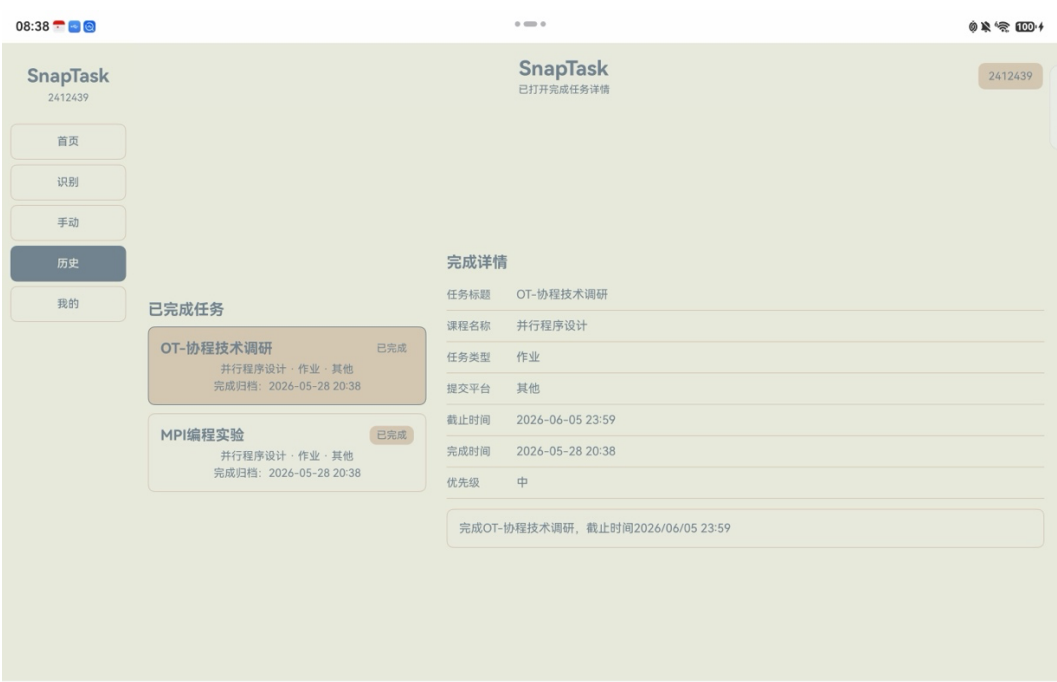


图 4.5 历史查看页效果图

如下图 4.6 所示，在我的页面里，可以查看自己的个人信息，填写 DeepSeek 的 api 用于 ai 解析，也可以清楚本人的所有数据，或者退出登录功能。



图 4.6 我的页效果图

第五章 总结与展望

第一节 全文总结

本项目完成了一个面向学生课程任务管理的 HarmonyOS 应用 SnapTask。系统以 ArkTS 和 ArkUI 构建前端界面，以 Spring Boot 提供后端接口，以 MySQL 保存业务数据，并结合本地 OCR 与 DeepSeek 大模型解析实现截图到 Todo 的自动转换。项目实现了登录注册、首页待办、手动添加、截图识别、AI 多任务解析、任务确认、任务删除、完成归档、历史详情、个人数据清理和横屏适配等功能。通过 MVVM 分层设计，前端状态和业务逻辑保持较清晰的组织；通过后端统一接口和数据库设计，任务数据能够稳定持久化。整体来看，项目达到了课程大作业对移动端界面、网络通信、数据库和综合应用能力的要求，也较好地贴合了学生课程任务管理的实际需求。

第二节 未来展望

后续可以从提醒能力、账号安全、AI 解析质量和部署方式四个方向继续完善。提醒能力方面，可以增加系统日历或通知提醒，在截止时间前自动提醒用户。账号安全方面，可以对密码进行加密存储，并增加登录态持久化和自动登录。AI 解析方面，可以继续优化提示词和字段校验规则，支持更多课程平台和更复杂的截图版式。部署方面，可以将后端服务部署到云服务器，避免平板和电脑必须处于同一 Wi-Fi 的限制，使 SnapTask 从课程项目进一步接近可长期使用的真实应用。

参考文献

- [1] 刘安战, 余雨萍, 陈争艳. HarmonyOS移动应用开发(ArkTS版)[M]. 北京: 清华大学出版社, 2023.
- [2] 马剑威, 贾振锋. 鸿蒙HarmonyOS NEXT开发之路 卷1: ArkTS语言篇[M]. 北京: 清华大学出版社, 2025.
- [3] 李智威. 鸿蒙ArkUI应用开发[M]. 北京: 清华大学出版社, 2026.

致谢

我非常荣幸能够在本学期选上了刘嘉欣老师，张彪老师，董前琨老师开设的移动应用开发课程。在专业课任务繁重的学期，能够修到一门可以任我发挥奇思妙想并付诸实践的课程，真的是一件幸事！

首先我要感谢三位老师的辛勤付出，上学期选修了刘嘉欣老师的 Java 课程，我就意识到刘老师对待教学极为认真的态度，所以这学期义无反顾的继续选修刘老师的课程。在这学期，我同样感受到了刘老师对课堂、对自己教授知识的认真态度，使我受益匪浅。而张彪老师的实验课，能手把着手教我们敲代码，带我们快速熟悉鸿蒙开发，绝对可以说的上是功不可没。而董前琨老师也使我印象深刻，自从上大学以来，我每学期都或选修或必修了董老师的课程，不论是大一的 C++，还是大二的 verilog，计算机组成原理。刚接触大学生活的见识了董前琨老师解答同学疑问时的耐心态度和亲和力，使我减少了对老师的畏惧情绪，能够更放开的和老师进行互动。

综上，非常感谢三位老师在这学期带来的精彩课程！